

EduGenie

AI-Powered Educational Assistant

Project Documentation

Project Description:

EduGenie is a lightweight, AI-powered educational assistant built to simplify and enhance the learning experience through the power of Generative AI. It is designed for students, self-learners, and educators who want quick, intelligent support across different subjects and academic levels.

Built with FastAPI on the backend and a responsive HTML, Tailwind CSS, and JavaScript frontend, EduGenie combines cloud-based generative AI (Google Gemini API, gemini-2.5-flash model) with a clean, interactive interface to deliver fast and accurate educational assistance. The platform brings together five core learning tools — Intelligent Q&A, AI-Powered Quiz Generation, Text Summarization, Personalized Learning Path generation, and Flashcard Generation — into a single, unified workspace, with activity history and theme preference remembered locally in the browser.

Scenarios

Scenario 1: A student preparing for an exam asks a conceptual question in the Intelligent Q&A module. EduGenie returns a clear, direct answer along with extra educational context — related concepts, examples, or background — helping the student build a deeper understanding rather than a one-line answer.

Scenario 2: A self-learner picks a topic and requests a quiz. The Quiz Generator produces a set of topic-specific multiple-choice questions, each with an explanation for the correct answer, allowing the learner to test their understanding and immediately see why an answer is right or wrong.

Scenario 3: A user pastes a long chapter of study material into the Summarizer and selects a “medium” length. EduGenie condenses the content into a concise, easy-to-review summary, saving the learner time before an exam.

Scenario 4: A beginner wants to learn a new subject from scratch. The Personalized Learning Path feature generates a structured Beginner → Intermediate → Advanced roadmap, giving the learner a clear sequence of topics to follow.

Scenario 5: Before a test, a student switches to the Flashcard Generator, which auto-generates flip-style flashcards for the chosen topic, enabling quick, active-recall revision. Dark mode and the activity history are remembered automatically across visits, since both are persisted locally in the browser.

Technical Architecture:

Description: EduGenie follows a two-tier client–server architecture. The frontend is a single-page interface built with HTML5, Tailwind CSS, and vanilla JavaScript, requiring no build step or framework. It communicates over REST endpoints with a FastAPI backend, which is organized into feature-specific routers (qa, quiz, summarize, learning_path, flashcards) that validate requests using Pydantic models and delegate the actual generation work to a shared Gemini API connector service (gemini_service.py). That service builds prompts, calls the Google Gemini API (gemini-2.5-flash), and returns structured responses back through the router to the frontend, which renders the results and persists history and theme preferences in the browser's localStorage.

(Note: EduGenie does not use a database — history and preferences are stored client-side in the browser, so no ER diagram is required for this project.)

Pre-requisites:

- **FastAPI Framework Knowledge:** [FastAPI Documentation](#)
- **Google Gemini API Familiarity:** [Gemini API Documentation](#)
- **HTML, Tailwind CSS, and JavaScript Skills:** [Tailwind CSS Documentation](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **Pydantic Fundamentals:** [Pydantic Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** [Uvicorn Documentation](#)

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate a Google Gemini API key and configure it securely in the project environment.
- **Activity 1.2:** Research and select the appropriate generative AI model from Gemini for educational content generation.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend, FastAPI backend, and Gemini integration.
- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for FastAPI and the Gemini API.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core functionalities: Intelligent Q&A, Quiz Generation, Text Summarization, Learning Path, and Flashcard Generation.
- **Activity 2.2:** Implement the FastAPI routers to manage endpoint handling and request validation, ensuring smooth API interactions with Gemini.

Milestone 3: Backend (main.py) Development

- **Activity 3.1:** Write the main application entry point in main.py, mounting each feature router and integrating AI responses.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using HTML, Tailwind CSS, and JavaScript, ensuring a responsive and intuitive layout with dark mode support.
- **Activity 4.2:** Create dynamic rendering logic in script.js to display results for each feature and persist history and theme in localStorage.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the environment and ensuring all dependencies are correctly installed.
- **Activity 5.2:** Run and verify the FastAPI backend locally using Uvicorn, and confirm the frontend communicates correctly with each endpoint.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, the focus is on selecting the appropriate generative AI model from Google Gemini for EduGenie's educational assistance needs. This involves researching the capabilities and performance of the available Gemini models, ensuring the chosen model aligns well with the application's objectives of answering questions, generating quizzes, summarizing text, building learning paths, and creating flashcards across different subjects and academic levels.

Activity 1.1: Generate a Gemini API Key

Before the application can communicate with the Gemini model, a valid Gemini API key is required. Follow the steps below to create one and connect it securely to the project.

- **Step 1 — Create a Google AI Studio Account:** Visit Google AI Studio and sign in with a Google account.
- **Step 2 — Navigate to API Keys:** From the Google AI Studio dashboard, open the “Get API key” section.
- **Step 3 — Create a New API Key:** Click “Create API Key” and choose or create a Google Cloud project to associate it with.
- **Step 4 — Copy the API Key:** Copy the generated key immediately and store it in a safe place, since it will not be shown again in full after the dialog closes.
- **Step 5 — Add the Key to Your Project:** Create a .env file inside the backend/ folder (if it does not already exist) and paste the key as shown below:

```
GEMINI_API_KEY=your_api_key_here
```

- **Step 6 — Verify the Key is Loaded:** The application loads this key automatically at startup using python-dotenv:

```
from dotenv import load_dotenv
load_dotenv()
api_key = os.environ.get("GEMINI_API_KEY")
```

- **Important — Never Commit Your API Key:** Add .env to the .gitignore file to prevent accidentally pushing the key to a public repository:

```
.env
```

Activity 1.2: Research and Select the Appropriate Generative AI Model

The following points were researched to select the best model for the project:

- **Understand the Project Requirements:** Review the specific needs of EduGenie, focusing on the types of content it will generate (answers, quizzes, summaries, learning paths, and flashcards).
- **Explore Gemini's Model Documentation:** Examine the available Gemini models, including their functionalities, context windows, speed, and limitations.
- **Evaluate Model Performance:** Compare models based on response quality, generation speed, and relevance to educational content generation tasks.
- **Select the Optimal Model:** Choose gemini-2.5-flash for its balance of response quality and low latency, suitable for interactive, real-time educational tools.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including the frontend (HTML, Tailwind CSS, JavaScript), FastAPI backend, routers, services layer, and Gemini API integration.
- **Detail Frontend Functionality:** Outline how users interact with the application — selecting a feature (Q&A, Quiz, Summarize, Learning Path, Flashcards), entering their input, and viewing AI-generated results.

- **Outline Backend Responsibilities:** Specify how each FastAPI router handles incoming requests, validates input using Pydantic models, and communicates with the shared Gemini service.
- **Describe AI Integration Points:** Define how `gemini_service.py` constructs prompts for each feature, calls the Gemini API, parses the response, and returns structured results to the corresponding router.

Activity 1.4: Set Up the Development Environment

- **Install Python:** Ensure a recent Python 3 version is installed on the machine.
- **Create a Virtual Environment:** Set up a virtual environment using `venv` to isolate project dependencies.

```
python -m venv venv venv\Scripts\activate
```

- **Install FastAPI and Uvicorn:** Use `pip` to install FastAPI along with the Uvicorn ASGI server.
- **Install the Gemini SDK:** Install the Google Generative AI library to interact with the Gemini API.
- **Install python-dotenv:** Install `dotenv` support for secure API key management.

```
pip install -r requirements.txt
```

- **Configure the API Key:** Create a `.env` file inside `backend/` and add the Gemini API key as shown in Activity 1.1.
- **Set Up Application Structure:** Create the project directory structure including the `app/routes`, `app/services`, and `frontend` folders, along with `main.py` and `requirements.txt`.

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

Implement the five core generation features — Q&A, Quiz Generation, Summarization, Learning Path, and Flashcards — each backed by a dedicated prompt sent to the Gemini API.

- **Intelligent Q&A:** Accepts a student's question and returns a clear, direct answer along with additional educational context to deepen understanding.
- **Quiz Generator:** Produces topic-specific multiple-choice questions, each with the correct answer and a short explanation.
- **Text Summarizer:** Condenses study material into short, medium, or detailed summaries based on the user's selection.
- **Learning Path Generator:** Builds a structured Beginner → Intermediate → Advanced roadmap for any given topic.
- **Flashcard Generator:** Auto-generates flip-style flashcards (front/back pairs) for quick, active-recall revision.

Activity 2.2: Implement the FastAPI Backend

Define Routes in FastAPI:

- Set up dedicated route modules in `app/routes/` for each feature: `qa.py`, `quiz.py`, `summarize.py`, `learning_path.py`, and `flashcards.py`.

Process User Input:

- Define Pydantic request models for each endpoint so incoming JSON payloads (topic, question, text, difficulty, etc.) are validated automatically.
- Return typed Pydantic response models so the frontend receives a predictable, well-structured JSON shape for every feature.

Integrate API Calls:

- Within each route handler, call into `gemini_service.py` to construct a feature-specific prompt and submit it to the Gemini API using the `gemini-2.5-flash` model.
- Parse and validate the model's response before returning it to the frontend, handling malformed or unexpected output gracefully.

Milestone 3: Backend (main.py) Development

Milestone 3 focuses on creating and refining the core logic of the `main.py` file, which serves as the entry point of the EduGenie application. In this milestone, each feature's router is mounted onto the FastAPI app, CORS and configuration are set up, and the Gemini service connector is wired in so every route can reliably call the AI model.

Activity 3.1: Writing the Main Application Logic in `main.py`

Define the Core App and Mount Routers:

- Instantiate the FastAPI app in `main.py` and include each feature router (`qa`, `quiz`, `summarize`, `learning_path`, `flashcards`) under a common API prefix.

Set Up the Gemini Service Connector:

- Centralize all Gemini API calls inside `app/services/gemini_service.py` so every route reuses the same client configuration, retry handling, and prompt-building conventions.

Configure Middleware and Environment:

- Load environment variables (`GEMINI_API_KEY`) at startup using `python-dotenv`, and configure CORS middleware so the HTML/JS frontend can call the API during local development.

Enable Interactive API Docs:

- FastAPI automatically exposes interactive documentation at `/docs`, allowing each endpoint to be inspected and tested directly in the browser.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a clean, responsive interface for EduGenie. This involves building a single-page layout with HTML and Tailwind CSS to present all five learning tools, and writing vanilla JavaScript to call the backend, render AI-generated content, and persist history and theme preferences in the browser.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the main HTML file (`frontend/index.html`) as the single-page interface, including a header, a feature selector for Q&A, Quiz, Summarize, Learning Path, and Flashcards, and a results area.
- Use semantic HTML elements to keep the structure organized and accessible.

Design a Responsive Layout with Tailwind CSS:

- Use Tailwind CSS via CDN to style the layout, avoiding a separate build step while keeping the interface responsive across desktop and mobile.
- Implement a dark mode toggle that switches theme classes and remembers the user's preference across visits.

Create Separate UI Components for Each Feature:

- **Q&A Panel:** Displays the question, the direct answer, and supporting educational context.

- **Quiz Panel:** Renders multiple-choice questions with selectable options and reveals explanations after answering.
- **Summary Panel:** Shows the generated summary at the selected length (short, medium, or detailed).
- **Learning Path Panel:** Displays the roadmap as three staged sections — Beginner, Intermediate, and Advanced.
- **Flashcard Panel:** Presents flip-style cards that reveal the answer side on click or tap.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

Handle Form Submission Asynchronously:

- Attach submit/click event listeners for each feature form and send the input as a JSON POST request to the corresponding FastAPI endpoint using the Fetch API.
- Show a loading indicator while waiting for the AI response, and disable the relevant action button to prevent duplicate submissions.

Render Results Dynamically:

- Implement rendering functions in script.js that dynamically create and inject DOM elements for answers, quiz questions, summaries, learning paths, and flashcards.

Persist History and Preferences:

- Save each generated result to the browser's localStorage under a History log, and restore theme (light/dark) preference automatically on page load.

Milestone 5: Deployment

In Milestone 5, the focus is on deploying the EduGenie application locally and verifying that the FastAPI backend and static frontend communicate correctly end to end.

Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a terminal in the backend/ directory.
- Create and activate a virtual environment, then install all required dependencies from requirements.txt.

```
cd backend python -m venv venv venv\Scripts\activate pip install -r requirements.txt
```

Configure Environment Variables:

- Create a .env file inside backend/ and add the Gemini API key. The app loads this automatically using python-dotenv at startup:

```
GEMINI_API_KEY=your_api_key_here
```

Activity 5.2: Local Testing and Verification

Start the FastAPI Development Server:

- Run the application locally using Uvicorn:

```
uvicorn app.main:app --reload
```

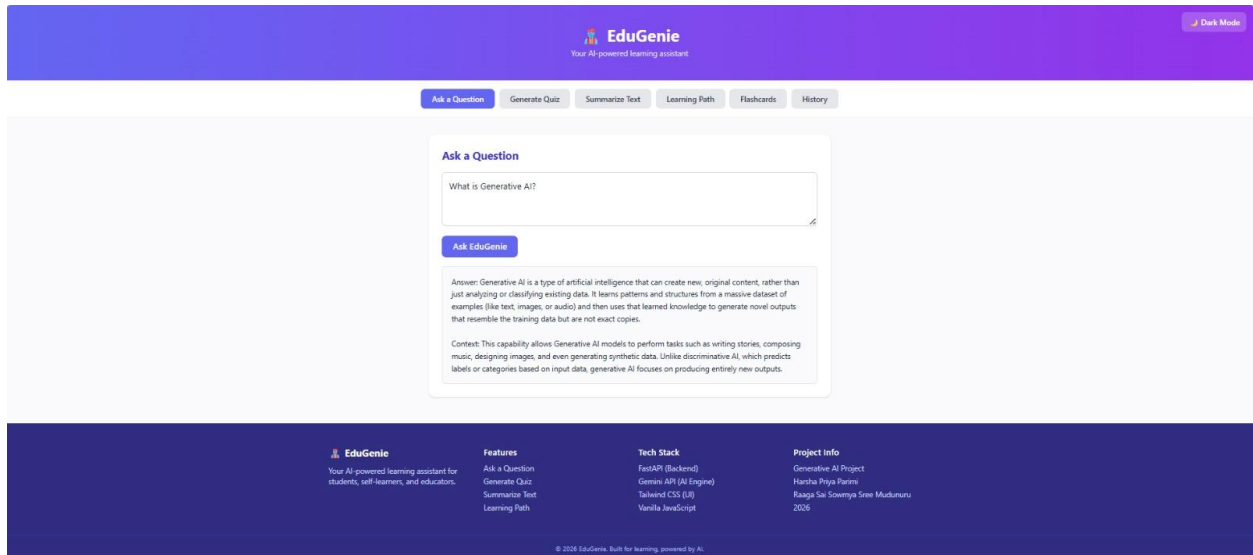
- Once running, the API is available at <http://127.0.0.1:8000>, with interactive documentation at /docs.
- Open frontend/index.html directly in a browser — no build step is required — and verify that all five features (Q&A, Quiz, Summarize, Learning Path, Flashcards) correctly call the backend and render results.

- Test the dark mode toggle and confirm the theme and activity history persist correctly after refreshing the page.

Exploring the Website's Web Pages:

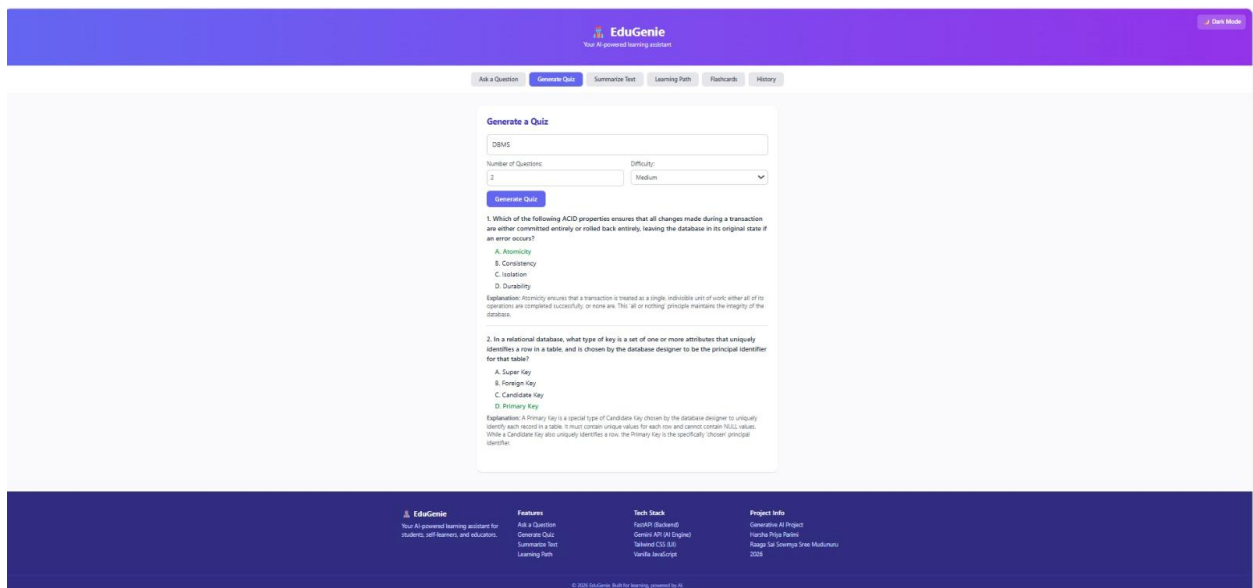
Home / Q&A Page:

Description: The landing view of EduGenie presenting the feature navigation and the Intelligent Q&A panel by default, where users type a question and receive a direct answer plus additional educational context.



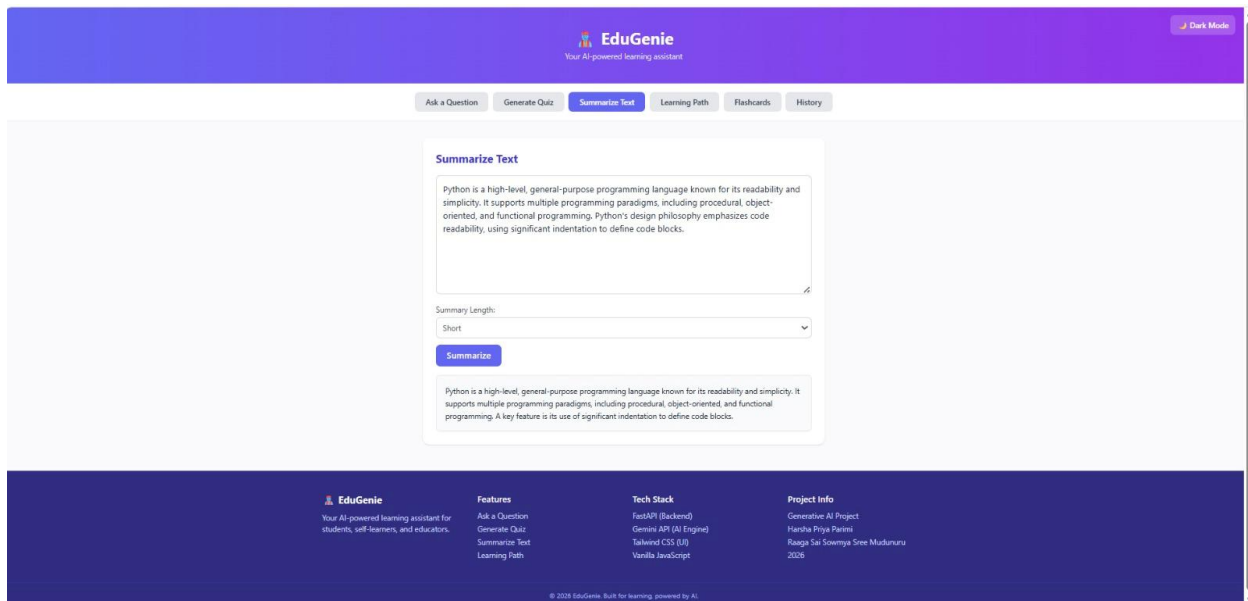
Quiz Generation Page:

Description: Users enter a topic and receive a set of AI-generated multiple-choice questions, each with an explanation revealed after the user selects an answer.



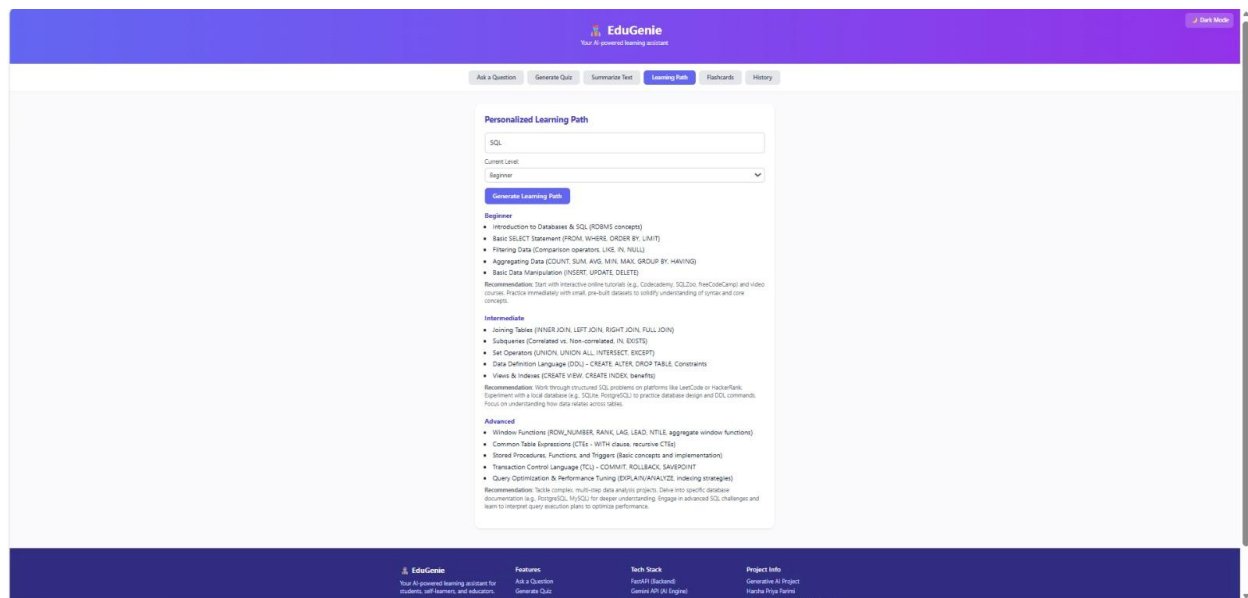
Summarize Page:

Description: Users paste study material and choose a summary length — short, medium, or detailed — and EduGenie returns a condensed version of the text.



Learning Path Page:

Description: Users enter a topic and receive a structured roadmap broken into Beginner, Intermediate, and Advanced stages.



Flashcards Page:

Description: Users enter a topic and receive a set of flip-style flashcards for quick revision, with each card revealing its answer on click or tap.

EduGenie
Your AI-powered learning assistant

Ask a QuestionGenerate QuizSummarize TextLearning PathFlashcardsHistory

Flashcard Generator

Number of Cards:

Generate Flashcards

What is Java?

What is the Java Virtual Machine (JVM)?

Explain 'Write once, run anywhere' (WORA) in Java.

What is an object in Java?

What is automatic Garbage Collection in Java?

EduGenie
Your AI-powered learning assistant for students, self-learners, and educators.

Features
Ask a Question
Generate Quiz
Summarize Text
Learning Path

Tech Stack
FastAPI (Backend)
Gemini API (AI Engine)
Tailwind CSS (UI)
Vanilla JavaScript

Project Info
Generative AI Project
Harsha Priya Parimi
Rasaga Sai Sowmya Sree Mudumuru
2026

© 2026 EduGenie. Built for learning, powered by AI.

History Mode:

Description: A history panel automatically lists past activity across all five features, stored locally in the browser via local Storage.

EduGenie
Your AI-powered learning assistant

Ask a QuestionGenerate QuizSummarize TextLearning PathFlashcardsHistory

Your HistoryClear History

FLASHCARDS
Java
5 flashcards generated12/7/2026, 5:51:54 pm

LEARNING PATH
SQL
3 stages generated12/7/2026, 5:50:51 pm

LEARNING PATH
SQL
3 stages generated12/7/2026, 5:50:08 pm

SUMMARIZE
Python is a high-level, general-purpose programming language...
Python is a high-level, general-purpose programming language known for its readability and simplicity. It...12/7/2026, 5:49:33 pm

QUIZ
DBMS
2 questions generated12/7/2026, 5:48:39 pm

QUIZ
DBMS
3 questions generated12/7/2026, 5:48:25 pm

QUIZ
Hima
3 questions generated12/7/2026, 5:48:02 pm

Q&A
What is Generative AI?
Answer: Generative AI is a type of artificial intelligence that can create new, original content, rather than j...12/7/2026, 5:45:14 pm

Conclusion:

EduGenie is a generative AI platform built with FastAPI and a Tailwind CSS frontend that streamlines everyday studying into five focused tools: Intelligent Q&A, Quiz Generation, Summarization, Learning Path creation, and Flashcards. It uses Google's Gemini API (gemini-2.5-flash) to instantly generate accurate, context-rich educational content, with activity history and theme preference persisted locally in the browser for a seamless experience across sessions. The project — which included model selection, FastAPI backend development, frontend design, and local deployment — highlights how a focused set of AI-powered tools can meaningfully support students, self-learners, and educators, with clear potential for future scalability.

Looking ahead, EduGenie offers significant opportunities for expansion, such as adding user accounts and server-side history storage, supporting multiple languages, enabling collaborative study sessions, and introducing spaced-repetition scheduling for flashcards. By continuing to refine prompt engineering and broaden frontend accessibility, the platform is well-positioned to evolve from a lightweight assistant into a comprehensive AI-powered study companion.

Contributors:

- Harsha Priya Parimi
- Raaga Sai Sowmya Sree Mudunuru

License:

This project was built for educational purposes as part of a Generative AI project.